

flocks 出网接管：安装、使用、排障

给跑 flocks 的内网机器 A 用，CentOS Stream 9 / RHEL 9 系（Rocky、Alma、Anolis 等），x86_64。装完以后 A 上所有程序（直接装的 flocks、跑在 Docker 里的 flocks 都算）访问公网时自动走客户已有的代理 B，访问内网直连，flocks 和其他程序不用改任何配置。B 上什么都不用装。

目录

- [1. 安装](#)
- [2. 使用](#)
- [3. 常见排障](#)
- [4. 必须知道的几条边界](#)

1. 安装

拿到的是一个文件 `flocks-egress-proxy-2.0.0-linux-x86_64.run`。三步：

1. 把这个文件放到机器 A 上（WinSCP / scp 拖到 root 的家目录就行）。
2. 在放文件的目录里执行一条命令（已经是 root 的话前面的 `sudo` 可以不写）：

```
sudo bash flocks-egress-proxy-2.0.0-linux-x86_64.run
```

它先找机器上现成的代理设置（dnf、`http_proxy` 环境变量、`wgetrc`），找到就直接用，屏幕上会打一行「用机器上已有的代理设置：... 来自 `/etc/dnf/dnf.conf`」。找不到就问你：

没有找到代理地址，请按提示输入。

代理地址（例如 `10.0.0.5:3128`，直接回车退出）：`10.0.0.5:3128`

代理账号（没有就直接回车）：

地址写「代理 IP:端口」就行；代理要账号密码的，在第二问填账号、第三问填密码（输入不回显）。

3. 等它跑完。最后几行是这样就装好了：

```
PASS 代理可用          http://10.0.0.5:3128: CONNECT
www.baidu.com:443 → 200
PASS 公网走代理        https://www.baidu.com/ http=200, 引擎日志:
```

www.baidu.com:443 → corp-proxy

WARN 内网直连 未设置 CHECK_INTRANET_URL, 跳过
(egress.conf 里填一个内网 HTTP 地址可自动验证)

PASS 10 / FAIL 0 / WARN 1

安装完成: 这台机器访问公网已经走 http://10.0.0.5:3128, 内网照常直连; flocks 照常使用, 不用改任何配置。

FAIL 0 就是好了; 那条 WARN 是可选的内网验证没配, 不是问题。有 FAIL、或者中途报错退出, 把屏幕上的内容整个发给技术支持, 或者对照第 3 节。

装完以后 flocks 照常装、照常用, 不用改任何配置, 也不用管先后顺序。只有一点: A 不能直连外网的话, 先装它再装 flocks, flocks 的安装脚本 (curl、git、uv、npm) 才能经代理下载。这个 .run 文件装完可以删, 也可以留着 (sudo bash xxx.run check | status | rollback 和装好后的 flocks-egress 命令等价)。

知道地址的话也可以直接跟在命令后面, 就不会问了:

```
sudo bash flocks-egress-proxy-2.0.0-linux-x86_64.run 10.0.0.5:3128
sudo bash flocks-egress-proxy-2.0.0-linux-x86_64.run 10.0.0.5:3128 --proxy-user 用户名 --proxy-password 密码
```

客户代理是哪种	地址怎么写
HTTP 代理 (最常见)	10.0.0.5:3128 (等于 http://10.0.0.5:3128)
HTTPS 代理 (到代理这一段是 TLS)	https://proxy.example.com:8443; 自签证书见排障
SOCKS5 代理	socks5://10.0.0.5:1080

装的时候会先真的经这个代理访问一次公网, 地址错、端口不通、账号不对会在这一步报错退出, 机器上什么都没改, 改对了重跑一遍就行。装的是两个开机自启的系统服务, 之后 flocks 怎么起、什么时候起都不用管它。A 只需要系统自带的 python3、nftables、systemd, 不用上外网装东西, 引擎程序在包里。同样内容还有一个 tar.gz 版本, 解压后 sudo bash setup-a.sh install, 效果一样。

2. 使用

装完不需要日常操作。flocks 照常 flocks start 即可; flocks 跑在 Docker 里也一样, 容器不用加任何参数。几条会用到的命令:

```
sudo flocks-egress check      # 验收: 代理通不通、公网是否走代理、内网是否直连
sudo flocks-egress status    # 服务状态、规则、上次验收结果
```

```
sudo flockes-egress install 10.0.0.6:3128 --proxy-user u --proxy-password p #
    换代理地址或账号（不给地址会问）
sudo flockes-egress rollback # 全部撤掉，机器恢复安装前的出网方式（参数文件改名保留）
journalctl -u flockes-egress -f # 看每一条出站连接：[TCP] 10.1.2.3:43166 -->
    api.openai.com:443 ... corp-proxy
```

引擎进程挂了 systemd 2 秒内自动拉起；引擎不在的那几秒公网直连（默认 ENGINE_DOWN=direct，能不能通看网络），回来后重新走代理。

需要改参数时编辑 /etc/flocks-egress/egress.conf，然后 sudo flockes-egress reconfigure。常用的几项（完整清单见包里的 egress.conf.example）：

参数	默认	什么时候改
DIRECT_CIDRS	空	内网用了 10/8、172.16/12、192.168/16 以外的地址段（比如自有公网段、DMZ），填进来直连，逗号分隔
PUBLIC_UDP	reject	发往公网的 UDP 默认立刻拒绝（代理带不了 UDP）。机器本身能直连公网 UDP、又需要它（比如公网 NTP），改 allow
ENGINE_DOWN	direct	引擎不在时默认直连兜底；要严格按代理策略、宁可断网也不许绕过，改 reject
CONTAINERS	auto	本机 Docker 容器的流量默认一起接管；不想管容器改 no
CHECK_INTRANET_URL	空	填一个内网 HTTP 地址，check 就会顺带验证内网直连

内网怎么算：目标 IP 落在 10/8、172.16/12、192.168/16、回环、链路本地、DIRECT_CIDRS 里就直连，其余算公网经代理。DNS 不用管：安装时自动探测内网 DNS 能不能解析公网域名，能就不碰 DNS，不能就由代理去解析。

3. 常见排障

现象	处理
「bash: flockes-egress-proxy-...run: No such file or directory」	命令不是在放文件的目录里执行的：cd 到那个目录（比如 cd /root），或者把文件路径写全

现象	处理
「请用 sudo 运行」	当前不是 root: 命令前面加 <code>sudo</code> , 或者先 <code>su -</code> 切到 root
安装时「代理测试失败: 连不上代理」	地址 / 端口错, 或 A 到 B 之间有防火墙。先在 A 上 <code>curl -x http://B:端口 https://www.baidu.com/</code> 确认通不通
「代理要求认证 / 账号密码不对 (HTTP 407) 」	加 <code>--proxy-user</code> / <code>--proxy-password</code> , 或核对密码
「代理拒绝 CONNECT ... (HTTP 403) 」, 或某类功能不通	B 的访问控制不放行这个目标或端口。squid 默认只允许 <code>CONNECT</code> 到 443; flocks 用到的其他端口要在 squid 里放开——80 (明文 HTTP) 加进 <code>SSL_ports</code> ; 993 / 465 (邮件渠道 IMAPS / SMTPS) 、22 (agent 用 SSH 连公网主机时) 这些小于 1025 的端口要同时加进 <code>Safe_ports</code> 和 <code>SSL_ports</code> 两个 <code>acl</code> , 只加一个仍是 403。 <code>journalctl -u flocks-egress -p warning</code> 能看到是哪个目标被拒
「代理的 TLS 证书校验失败」	https 代理用了企业自签证书: <code>cp 根证书.crt /etc/pki/ca-trust/source/anchors/</code> && <code>update-ca-trust</code> , 再装; 临时可加 <code>--proxy-tls-insecure</code>
<code>check</code> 里「公网走代理」FAIL, 提示已进引擎但请求失败	B 拒绝了这个目标, 或 B 自己出不去。 <code>journalctl -u flocks-egress -p warning</code> 看 <code>dial corp-proxy ... error</code>
<code>check</code> 里「DNS 不接管」FAIL, 解析失败	内网 DNS 变了。 <code>sudo flocks-egress reconfigure</code> 让它重新探测
某个内网服务被送去了代理	它的地址不在内置内网段里: 把网段填进 <code>DIRECT_CIDRS</code> 后 <code>reconfigure</code>
flocks 报公网请求失败, 但 <code>check</code> 全过	看 <code>journalctl -u flocks-egress -f</code> 里那条连接的结果; 多半是 B 对该目标的限制
服务起不来	<code>journalctl -u flocks-egress -n 50</code> 。常见是端口被占 (改 <code>egress.conf</code> 里的 <code>REDIR_PORT</code> 等)
机器上有 <code>nftables.service</code> , <code>restart</code> 它之后公网不通	本工具装了 drop-in 会自动把规则加回来; 没恢复就 <code>systemctl restart flocks-egress-rules</code>

现象	处理
装过一半，报「检测到上次安装的残留」	<code>sudo bash flocks-egress-proxy-2.0.0-linux-x86_64.run rollback</code> 清掉再装
想临时全部直连	<code>sudo systemctl stop flocks-egress-rules</code> （连引擎一起停）；恢复 <code>sudo systemctl start flocks-egress-rules flocks-egress</code>

4. 必须知道的几条边界

- 接管的是 A 上所有进程和本机 Docker（bridge 网络）容器发出的公网连接，含 root 的 dnf、ssh 出站。能不能通取决于 B 允许 `CONNECT` 到哪些端口。
- 公网 UDP 不走代理（代理带不了），默认拒绝：chrony 对公网 NTP 池会立刻失败，请指向内网 NTP。公网 IPv6 同样被拒，程序会回落 IPv4。
- A 的网卡本身是网桥（虚拟机宿主的 br0）时会自动识别为上联口、不当容器处理；之后改了网络拓扑要 `sudo flocks-egress reconfigure`，`check` 发现不一致会提醒。
- 引擎的普通代理口 7890 只在本机 `127.0.0.1` 上，没有账号，A 上的本地用户可以借它上网；介意的话改 `MIXED_PORT` 指到一个被本机防火墙拦住的端口。
- 引擎是 mihomo v1.19.31（GPL-3.0），以独立进程、低权限用户运行，见 `THIRD_PARTY_NOTICES.md`。